

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Felipe Yuzo Tanaka Shida

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	3
4	Atividades realizadas	3
4.1	Atividade 2 — Catálogo de filmes — Tom Hanks	3
4.2	Atividade 3 — Desacoplando o login — microsserviço de autenticação	5
4.3	Atividade 4 — Controle de acesso por papel — RBAC	7
4.4	Atividade 5 — Logs e auditoria	9
4.5	Atividade 6 — Upload e perfil de usuário	11
5	Considerações finais	12
6	Declaração de autoria	13

1 Introdução

Este relatório documenta as atividades práticas desenvolvidas na disciplina ISW055. Introdução à Computação em Nuvem, ministrada pelo Prof. Allan Siriani. A disciplina propõe capacitar os alunos nos fundamentos práticos da computação em nuvem moderna, abordando arquiteturas baseadas em microsserviços, containerização com Docker, isolamento de redes internas, controle de acesso e autorização (RBAC), governança por meio de logs de auditoria e segregação de persistência entre bancos relacionais e sistemas de armazenamento de objetos (Object Storage).

O fio condutor do bimestre foi o desenvolvimento e a evolução arquitetural progressiva de uma aplicação web de Catálogo de Filmes voltada à filmografia do ator Tom Hanks, com integração à API pública da TMDb (The Movie Database). Ao longo das etapas do curso, o sistema evoluiu de uma estrutura inicial simples para uma arquitetura distribuída e resiliente, composta por microsserviços independentes e orquestrados via Docker Compose: um serviço principal de catálogo, um microsserviço isolado de autenticação (auth-service) com envio de e-mails transacionais e recuperação de senha, um microsserviço de auditoria (log-service) baseado em Redis Streams, persistência relacional no MariaDB e gerenciamento de arquivos binários e fotos de perfil no MinIO (compatível com a API AWS S3).

Este documento reúne a fundamentação teórica, as decisões de engenharia e os resultados práticos de todas as fases do projeto. O relatório está organizado em seções que detalham a arquitetura geral do sistema, os modelos de dados e fluxos de segurança (como autenticação JWT e ciclo de vida de tokens de redefinição de senha), a matriz de permissões e controle de acesso baseado em papéis (RBAC), a estratégia de auditoria não bloqueante com Redis Streams, o armazenamento de mídia e prevenção de vulnerabilidades (MIME spoofing e IDOR) no MinIO, finalizando com roteiros de teste passo a passo via terminal (cURL) e as respectivas evidências de funcionamento.

2 Metodologia

Todas as atividades foram desenvolvidas no mesmo repositório público do GitHub, como continuação de um único projeto (catálogo de filmes). A data e a hora de cada entrega foram extraídas do histórico de commits (git log) e conferidas na página do commit no GitHub. Os prints mostram o README com a menção ao professor, o histórico de commits e o sistema em execução.

As atividades foram desenvolvidas em ambiente local e de homologação utilizando containers orquestrados com Docker e Docker Compose, integrando microsserviços em Node.js (Express), banco relacional MariaDB, mensageria de auditoria via Redis Streams (audit:events), armazenamento de objetos compatível com S3 via MinIO e envio de e-mails transacionais (Mailtrap/Brevo), além da integração com a API da TMDb. O deploy e gerenciamento em produção foram realizados via Portainer. As referências e fontes de consulta incluíram as documentações oficiais das ferramentas (Docker, Redis, MinIO S3 API, Express) e os roteiros da disciplina ISW055 (Prof. Allan Siriani).

As evidências foram coletadas a partir da execução de roteiros de testes de ponta a ponta via cURL (validando fluxos de autenticação, controle de acesso RBAC com status 403 Forbidden e 200 OK, proteção contra falhas de IDOR e inspeção de Magic Bytes no upload de imagens), além de consultas diretas à CLI do Redis (XRANGE), inspeção dos buckets no MinIO e logs dos containers no Portainer. Por fim, as datas, horários e marcos de entrega foram obtidos a partir do histórico cronológico de commits do repositório Git (git log), registros de deploy na stack do Portainer e timestamps auditados gerados pelas requisições.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.	20/08/2026	22/08/2026 15:28	entregue com atraso- deixei o projeto aberto e tinha pensado que já havia feito o commit.
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	27/08/2026 14:26	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	03/09/2026 14:43	entregue
5	Logs e auditoria	microserviço de auditoria (log-service) que registra no Redis Streams quem fez cada ação (login, favoritos, comentários e acessos negados), quando e de qual IP, com consulta restrita a administradores via GET /api/logs.	25/09/2026	23/09/2026 16:05	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	30/09/2026 15:47	entregue

4 Atividades realizadas

4.1 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	22/08/2026 15:28
Situação	entregue com atraso- deixei o projeto aberto e tinha pensado que já havia feito o commit.
Evidência	GitHub — commit 903137f + README + print do catálogo
Link	https://github.com/FeYuzo/catalogo-de-filmes-api/commit/903137fb2114ef5abe23a3aad9bf5de3d463756

Nesta atividade inicial, foi desenvolvida a versão base da aplicação web de Catálogo de Filmes dedicada à filmografia do ator Tom Hanks. No frontend, construiu-se uma interface responsiva em JavaScript Vanilla, HTML5 e CSS3 com busca, visualização de detalhes, listagem de elenco, trailers e interação com a comunidade. No backend em Node.js (Express), estruturou-se uma API RESTful organizada em arquitetura em camadas (rotas, controllers, middlewares e serviços) com persistência relacional no MariaDB para usuários, filmes favoritos e comentários, além da orquestração inicial do ambiente em containers utilizando Docker e Docker Compose.

Entre as principais decisões técnicas, optou-se por encapsular as requisições à API da TMDB exclusivamente no backend, protegendo a chave de API contra exposição no cliente e padronizando as respostas consumidas pelo frontend. Para a segurança e integridade dos dados, adotou-se autenticação stateless com JWT (JSON Web Tokens) e criptografia de senhas com Bcrypt, garantindo que favoritos e comentários fossem estritamente isolados por `usuario_id`. Além disso, implementou-se a criação automática das tabelas relacionais na inicialização do banco (`db.js`) com integridade referencial (`ON DELETE CASCADE`), facilitando o ciclo de desenvolvimento e preparando a aplicação para as futuras etapas de desacoplamento em microsserviços.



```
Windows PowerShell
PS C:\Users\shida\OneDrive\Documentos\atividade-filmes-main> git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad" 903137f
903137f 22/08/2026 15:28
PS C:\Users\shida\OneDrive\Documentos\atividade-filmes-main> |
```

Figura 1: Atividade 2 — evidência da entrega (link, data e hora visíveis).

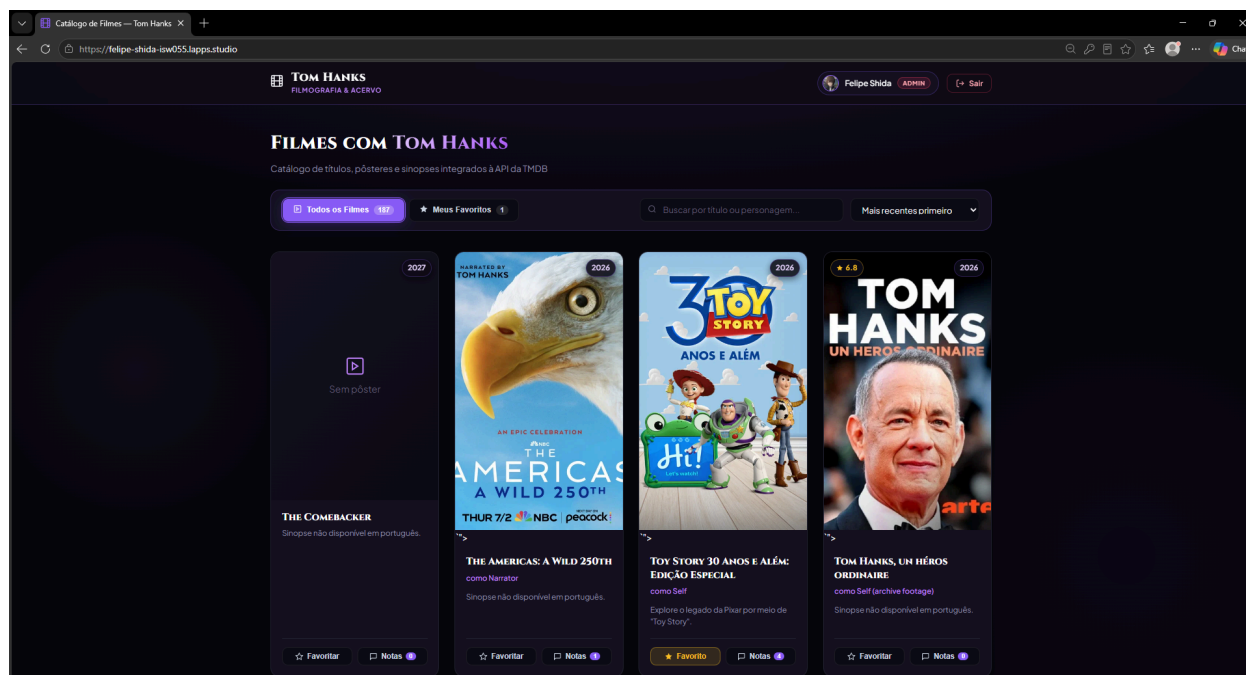


Figura 2: Atividade 2 — Catálogo de Filmes com Tom Hanks em execução no navegador.

A principal dificuldade inicial esteve na sincronização da inicialização do container MariaDB com a API Node.js no Docker Compose, evitando erros de recusa de conexão antes do banco estar pronto, o que foi resolvido estruturando um pool de conexões resiliente com tentativas de reconexão e verificação automática de schema no db.js. Além disso, a heterogeneidade e a presença de campos nulos nos retornos da API da TMDb (como ausência de pôsteres ou trailers) foram contornadas criando rotinas de sanitização, fallbacks visuais e mapeamento centralizado de dados no tmdbService.js.

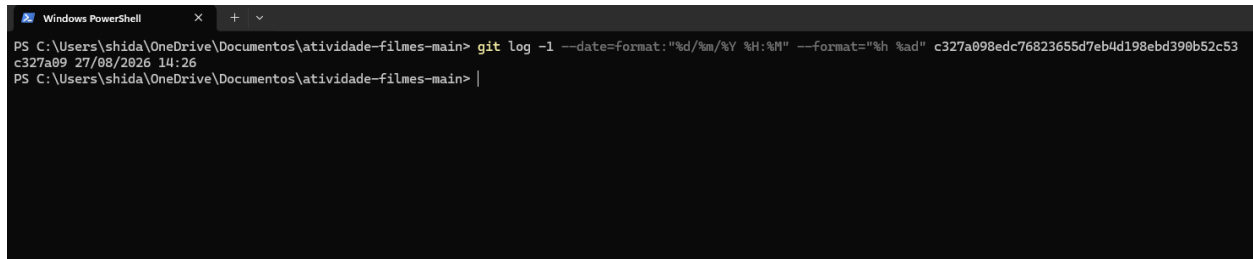
4.2 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	27/08/2026 14:26
Situação	entregue
Evidência	GitHub — commit c327a09 + docker-compose.yml + print do login funcionando
Link	https://github.com/FeYuzo/catalogo-de-filmes-api/commit/c327a098edc76823655d7eb4d198ebd390b52c53

Nesta atividade, a arquitetura foi evoluída para um modelo de microsserviços com a extração e criação de um serviço dedicado de autenticação (auth-service). Esse novo serviço assumiu integralmente a gestão de credenciais, cadastro com papéis (roles usuario e admin), emissão de tokens JWT e a implementação completa do fluxo de recuperação de senha (“Esqueci minha senha”). Para isso, implementou-se a tabela reset_tokens no MariaDB e o envio real de e-mails transacionais formatados em HTML via SMTP (mailService.js) integrando com Mailtrap e Brevo, enquanto o catálogo principal passou a atuar como ponto de entrada consumindo o serviço de autenticação internamente.

A principal decisão arquitetural foi isolar o auth-service estritamente na rede interna do Docker, sem expor portas públicas para a internet no docker-compose.yml, fazendo com que o catálogo seja o único

canal de comunicação pública e diminuindo a superfície de vulnerabilidade. No aspecto de segurança, optou-se por gerar tokens de redefinição criptograficamente seguros (crypto.randomBytes) com expiração real estrita de 30 minutos e controle de uso único (usado = TRUE), garantindo proteção contra ataques de repetição e assegurando a separação de responsabilidades (Single Responsibility Principle) entre a regra de negócio do catálogo e o ciclo de vida de identidade dos usuários.



```
Windows PowerShell
PS C:\Users\shida\OneDrive\Documentos\atividade-filmes-main> git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad" c327a098edc76823655d7eb4d198ebd390b52c53
c327a09 27/08/2026 14:26
PS C:\Users\shida\OneDrive\Documentos\atividade-filmes-main> |
```

Figura 3: Atividade 3 — evidência da entrega (link, data e hora visíveis).

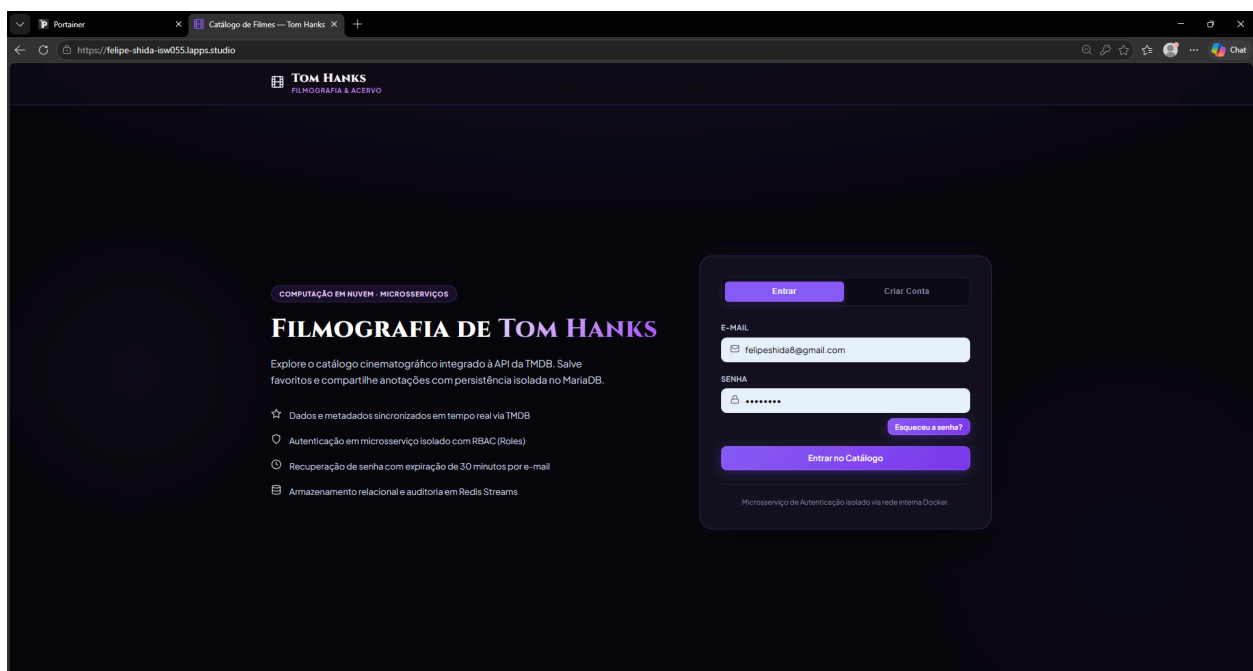


Figura 4: Atividade 3 — tela de login.

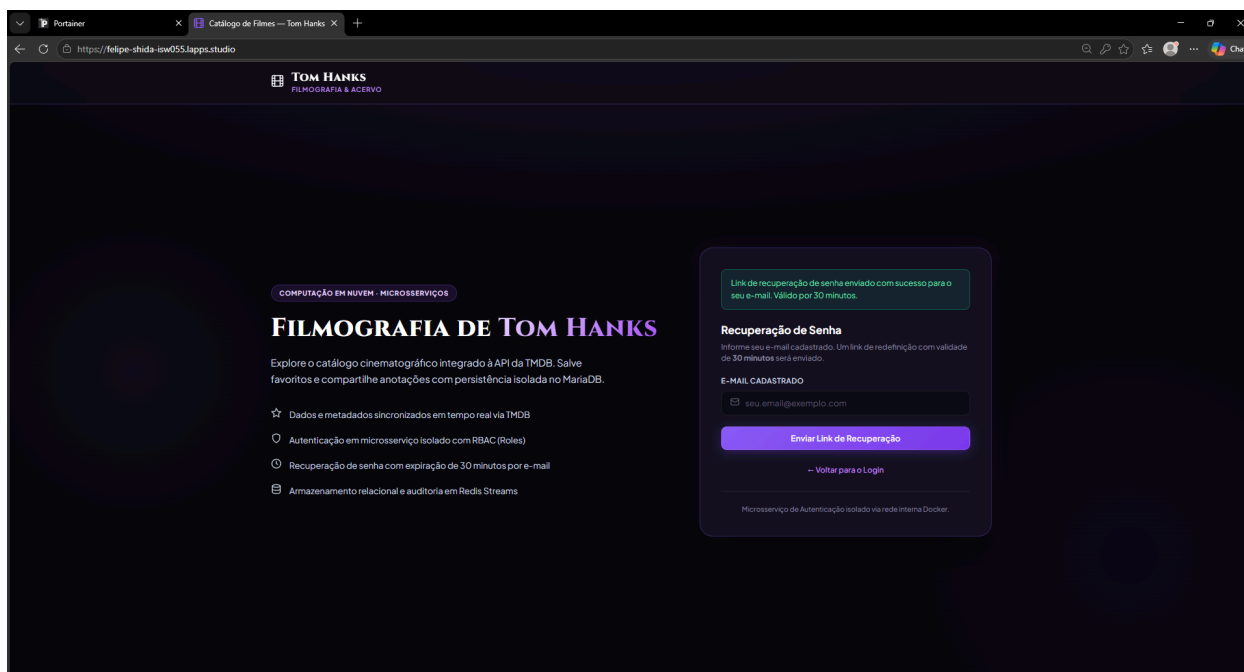


Figura 5: Atividade 3 — tela de “esqueci minha senha”.

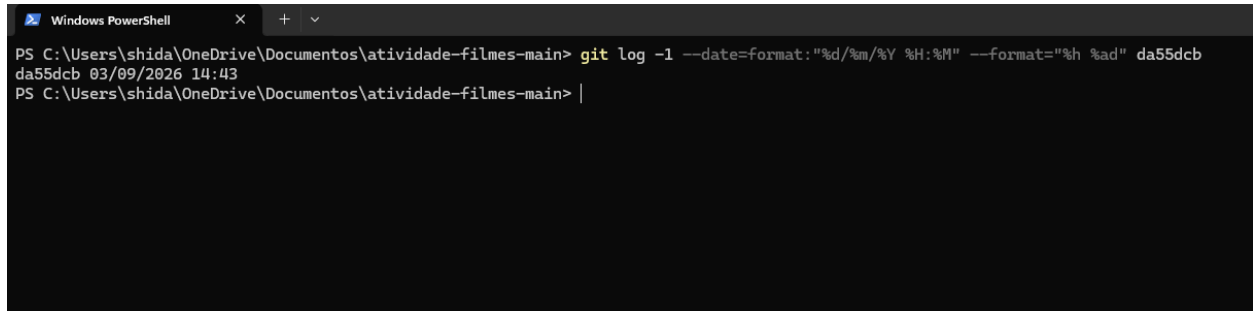
A principal dificuldade esteve na diferenciação entre a comunicação inter-serviços na rede interna do Docker (<http://auth-service:4000>) e a geração do link público de recuperação de senha enviado por e-mail, o que foi resolvido segregando as variáveis de ambiente `AUTH_SERVICE_URL` (para chamadas internas no container) e `APP_URL` (para montagem dos links acessíveis no navegador). Além disso, o tratamento de fuso horário e a consistência na expiração dos tokens no MariaDB foram solucionados utilizando comparações diretas de data no banco (`expira_em > NOW()`) e invalidando o token de forma atômica no momento da redefinição para impedir reuso.

4.3 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	03/09/2026 14:43
Situação	entregue
Evidência	GitHub — commit da55dcb + print do 403 e da ação de admin
Link	https://github.com/FeYuzo/catalogo-de-filmes-api/commit/da55dcbfda14bd5c484c62569099e76a2ffb2e1c

Nesta atividade, foi implementado o modelo de Controle de Acesso Baseado em Papéis (RBAC — Role-Based Access Control) com autorização real no servidor. Estruturou-se uma matriz de permissões granulares no formato `(permissions.js)`, definindo as capacidades dos papéis `usuario` e `admin`. Como ação restrita de negócio, implementou-se a Moderação de Comentários (`DELETE /api/comments/:id`), onde usuários comuns têm permissão apenas para excluir seus próprios comentários (`comentarios:excluir proprio`), enquanto administradores possuem a permissão exclusiva de moderação para apagar comentários de qualquer usuário (`comentarios:excluir qualquer`), respondendo com HTTP 403 Forbidden para tentativas não autorizadas e HTTP 200 OK para ações legítimas.

A principal decisão de engenharia foi a adoção do Padrão A (Enforcement Centralizado) em detrimento do Padrão B (permissões embutidas nas claims do JWT). Por meio desse modelo, o Catálogo consulta ativamente a autoridade centralizadora (authService.js) a cada operação sensível, garantindo consistência imediata e revogação instantânea de privilégios caso um usuário seja rebaixado ou bloqueado no banco de dados, eliminando janelas de vulnerabilidade causadas por tokens stateless desatualizados. Além disso, documentou-se a matriz de permissões completa e o comparativo técnico de trade-offs entre latência de rede e segurança imediata.



```
Windows PowerShell
PS C:\Users\shida\OneDrive\Documentos\atividade-filmes-main> git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad" da55dcb
da55dcb 03/09/2026 14:43
PS C:\Users\shida\OneDrive\Documentos\atividade-filmes-main> |
```

Figura 6: Atividade 4 — evidência da entrega (link, data e hora visíveis)

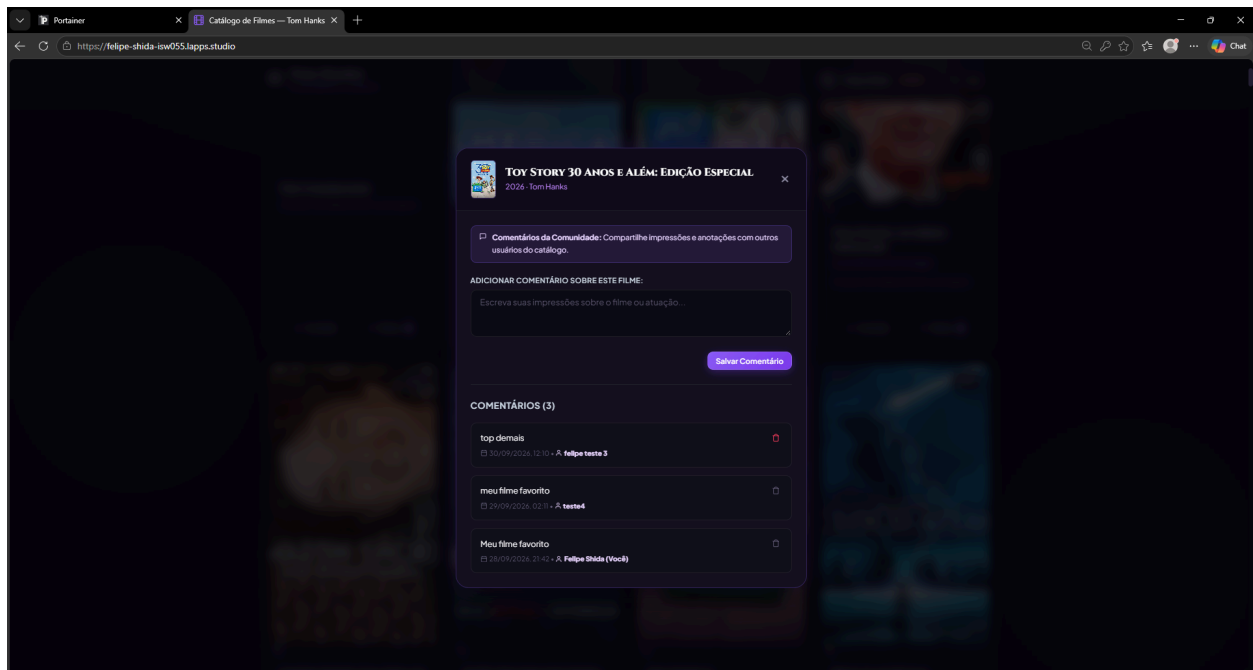


Figura 7: Atividade 4 — usuários com role ADMIN, conseguem excluir comentários de usuários comum

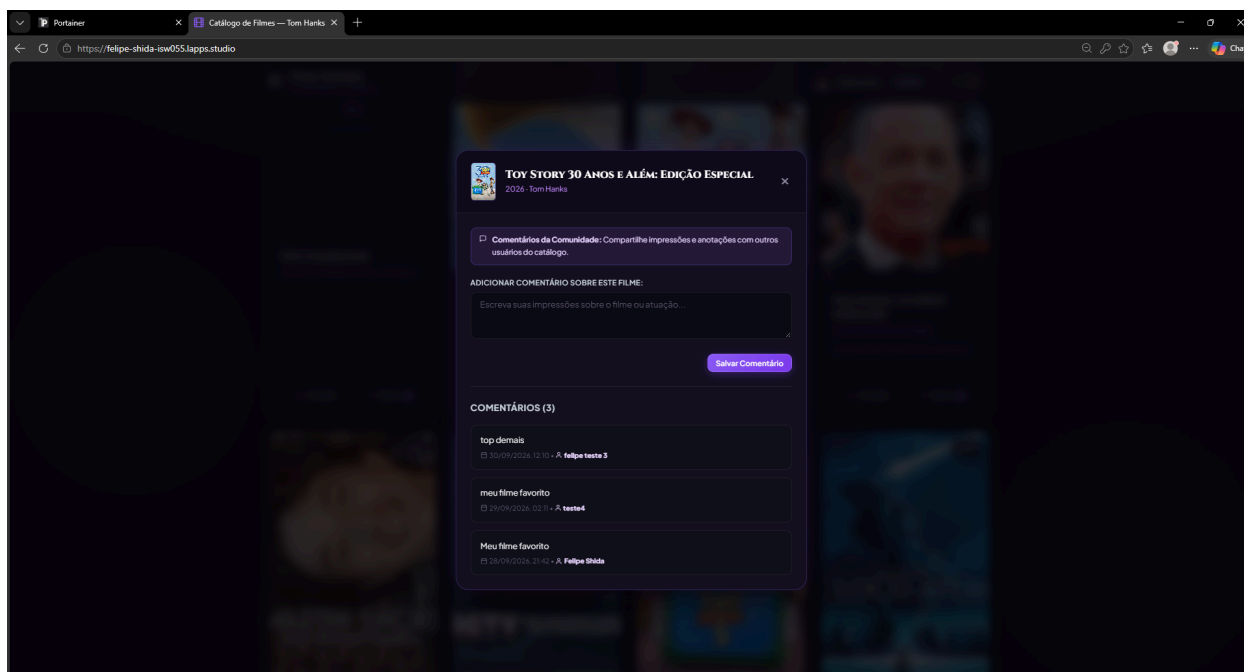


Figura 8: Atividade 4 — usuários com role comum, não conseguem excluir nenhum comentário

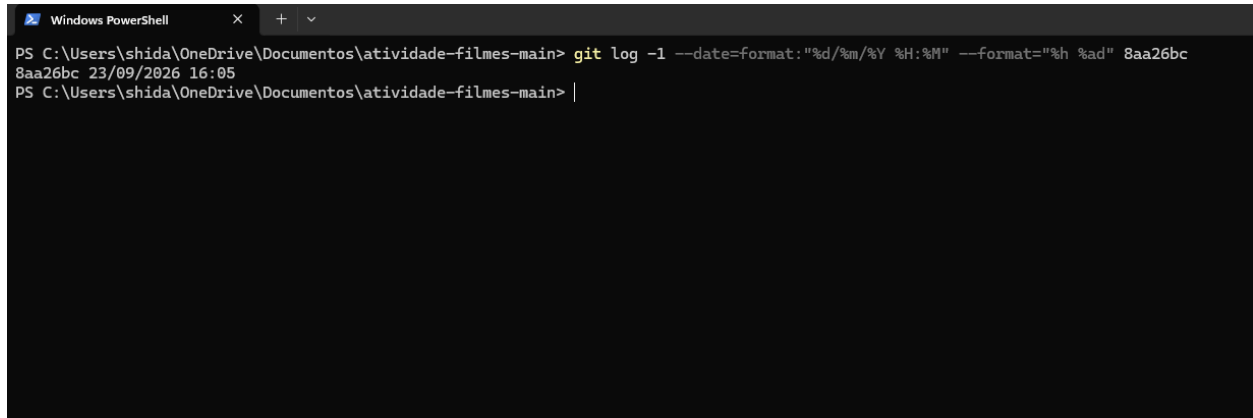
A principal dificuldade esteve na validação contextual da rota de exclusão de comentários (DELETE /api/comments/:id), onde a mesma URL precisava atender dois fluxos distintos: a autoexclusão pelo próprio autor e a moderação de conteúdo de terceiros por administradores. A solução foi estruturar o commentController.js com checagem em duas etapas, verificando primeiro a titularidade do comentário (usuario_id) e, em caso de autor diferente, disparando a verificação centralizada da permissão comentarios:excluir_qualquer junto ao auth-service, garantindo o retorno semântico correto entre 401 Unauthorized (não autenticado) e 403 Forbidden (papel insuficiente)

4.4 Atividade 5 — Logs e auditoria

Descrição	microserviço de auditoria (log-service) que registra no Redis Streams quem fez cada ação (login, favoritos, comentários e acessos negados), quando e de qual IP, com consulta restrita a administradores via GET /api/logs.
Data planejada	25/09/2026
Data realizada	23/09/2026 16:05
Situação	entregue
Evidência	GitHub — commit 8aa26bc + print da consulta de logs pelo admin
Link	https://github.com/FeYuzo/catalogo-de-filmes-api/commit/8aa26bc2c786b6f27bfaffe12a58fa24cc284584

Nesta atividade, foi implementado um microserviço dedicado e isolado de Logs e Auditoria (log-service), integrado a uma instância do Redis orquestrada via Docker Compose com durabilidade AOF. O serviço é responsável por persistir e gerenciar trilhas de auditoria imutáveis no fluxo temporal Redis Streams (audit:events) utilizando o comando XADD. Todos os eventos comportamentais e de segurança dos demais serviços foram instrumentados incluindo login, logout, favoritar_filme, comentar, apagar_comentario_moderacao e detecção de tentativas não autorizadas (permissao_negada), além da disponibilização da rota protegida GET /api/logs no Catálogo, restrita a administradores e suportando consultas cronológicas com XREVRANGE e XRANGE.

A principal decisão arquitetural foi a escolha técnica de Redis Streams em detrimento de tabelas no MariaDB e de listas simples (LPUSH). Essa decisão preservou o banco relacional de cargas massivas de escrita (write-heavy, read-rarely), garantindo ordenação temporal estrita com IDs monotônicos gerados nativamente pelo Redis e consultas por faixa de tempo em alta performance via Radix Trees. Adicionalmente, adotou-se o padrão de despacho assíncrono e não-bloqueante (fire-and-forget) nos serviços emissores (auditService.js), assegurando tolerância a falhas: mesmo em caso de instabilidade eventual do Redis ou do serviço de logs, a experiência e a requisição do usuário final nunca são travadas ou interrompidas.



```
Windows PowerShell
PS C:\Users\shida\OneDrive\Documentos\atividade-filmes-main> git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad" 8aa26bc
8aa26bc 23/09/2026 16:05
PS C:\Users\shida\OneDrive\Documentos\atividade-filmes-main> |
```

Figura 9: Atividade 5 — evidência da entrega (link, data e hora visíveis)

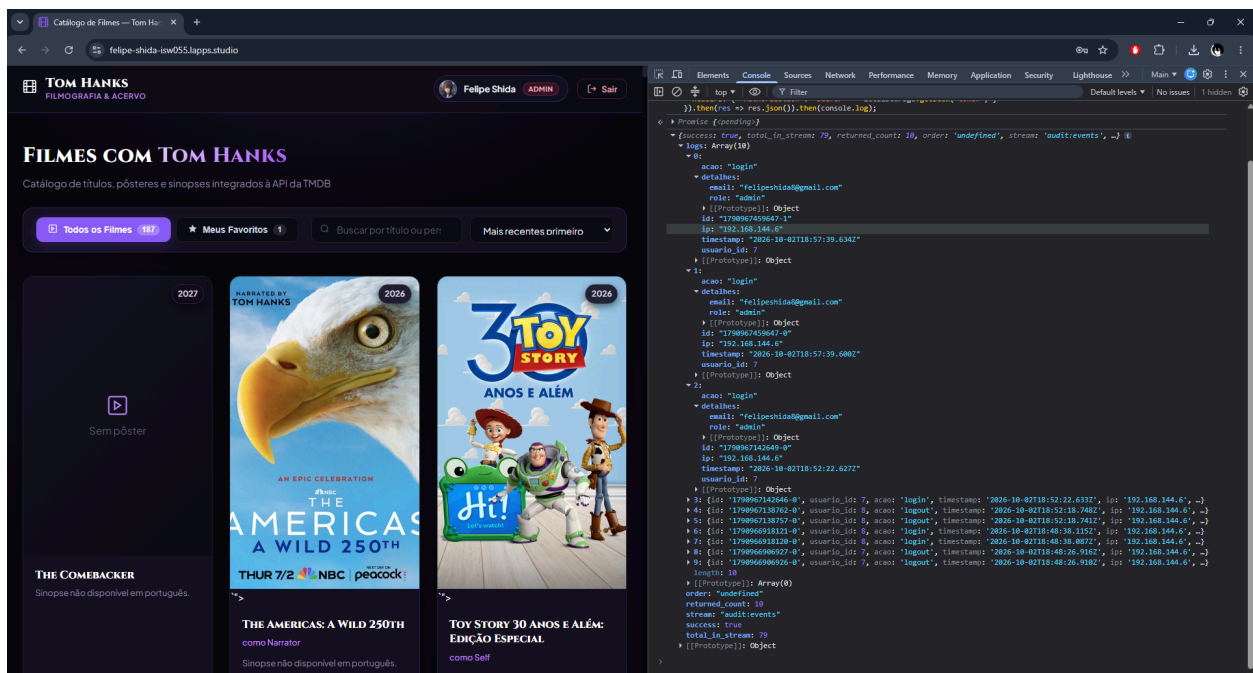


Figura 10: Atividade 5 — resultado mostrando todos os acessos, horário que acessou e ip

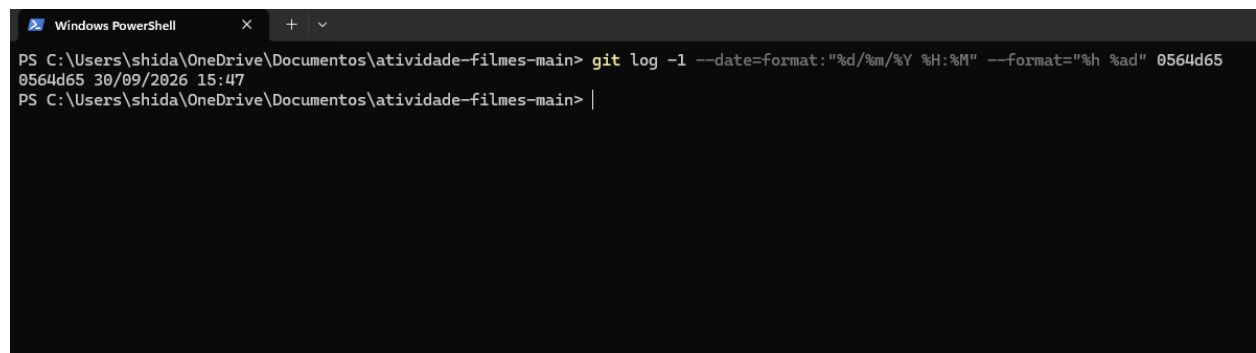
A principal dificuldade esteve na manipulação e conversão da estrutura de dados bruta do Redis Streams, que devolve pares chave-valor em vetores planos de strings e exige tratamento na serialização/desserialização de metadados complexos (detalhes). A solução foi desenvolver utilitários de parsing e formatação no logController.js para padronizar as respostas em JSON legível, além de isolar todos os despachos de auditoria no auditService.js sob o padrão assíncrono com tratamento resiliente de erros (try/catch não-bloqueante), impedindo que indisponibilidades pontuais do Redis afetassem a resposta HTTP ao usuário final.

4.5 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	30/09/2026 15:47
Situação	entregue
Evidência	GitHub — commit 0564d65 + print do perfil com foto
Link	https://github.com/FeYuzo/catalogo-de-filmes-api/commit/0564d65bf7043615ab88715ee0a632bc0d5668eb

Nesta atividade, foi desenvolvida a funcionalidade de página e modal de perfil de usuário com suporte a biografia personalizada, listagem de favoritos e upload de avatar, integrando o sistema a uma instância de Object Storage de alta performance através do MinIO (100% compatível com a API AWS S3) via minio.js. No backend, implementaram-se endpoints de perfil (profileRoutes.js) com processamento de arquivos em memória (multer.memoryStorage()), validação binária de segurança contra MIME-Type spoofing via Magic Bytes (fileValidation.js) para formatos JPEG, PNG, WebP e GIF, proteção estrita contra falhas de IDOR e geração automática de eventos de auditoria no Redis Streams (upload_foto_perfil, perfil_atualizado e permissao_negada).

A principal decisão de engenharia foi a segregação absoluta entre dados relacionais e arquivos binários, rejeitando o uso de tipos BLOB no MariaDB para evitar a degradação da memória cache (InnoDB Buffer Pool), fragmentação de páginas de disco e lentidão em backups. O MariaDB passou a armazenar apenas a chave textual do objeto, enquanto o binário reside no MinIO no bucket dedicado perfil-usuarios. Adicionalmente, optou-se por uma estratégia de Leitura Pública no bucket combinada a um Streaming Proxy no Catálogo (GET /api/profile/photo/'), permitindo que as fotos de perfil sejam entregues eficientemente pela mesma porta pública da aplicação (3000), sem necessidade de expor a porta direta do MinIO à internet.



```

Windows PowerShell
PS C:\Users\shida\OneDrive\Documentos\atividade-filmes-main> git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad" 0564d65
0564d65 30/09/2026 15:47
PS C:\Users\shida\OneDrive\Documentos\atividade-filmes-main> |
  
```

Figura 11: Atividade 6 — evidência da entrega (link, data e hora visíveis)

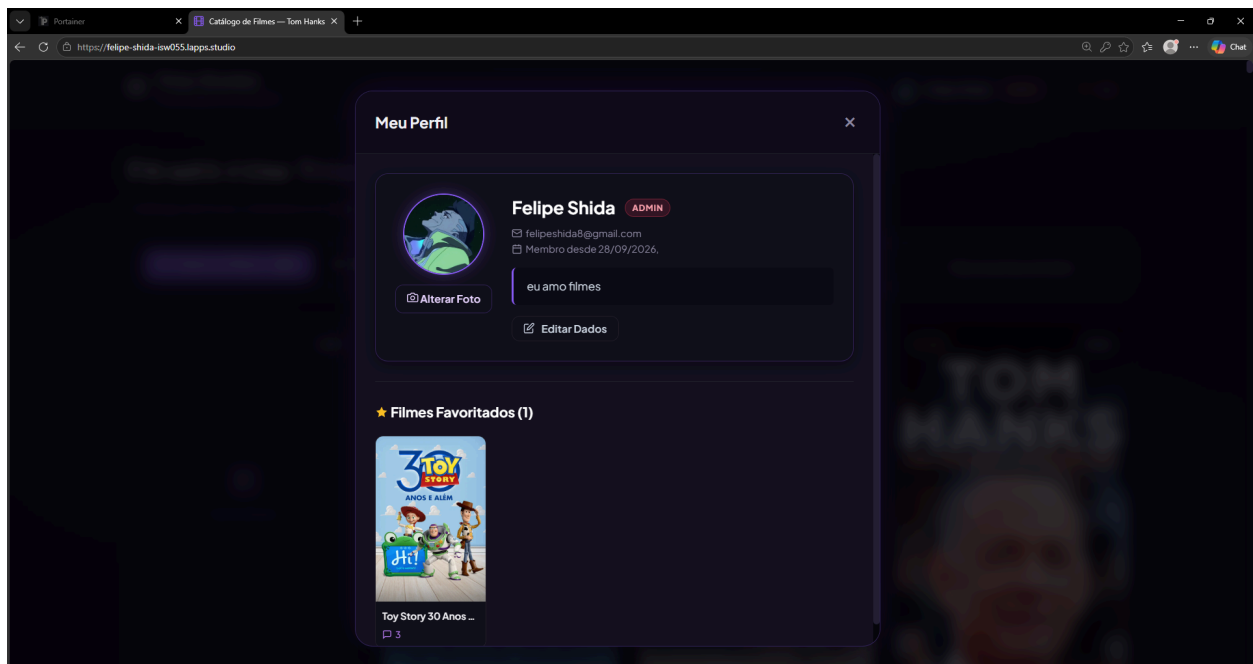


Figura 12: Atividade 6 — Perfil com foto de usuário, usando MinIO.

A principal dificuldade esteve na inicialização determinística do cliente MinIO e na aplicação de políticas de acesso ao bucket (perfil- usuarios) antes do recebimento de requisições, o que foi resolvido criando uma rotina de bootstrap assíncrona no minio.js que verifica a existência do bucket e injeta automaticamente a política JSON de leitura pública caso necessário. Além disso, a validação de segurança para impedir arquivos maliciosos forjados no cabeçalho HTTP foi solucionada inspecionando diretamente as assinaturas hexadecimais nativas nos primeiros bytes do buffer em memória (fileValidation.js), garantindo que apenas imagens autênticas fossem repassadas ao Object Storage.

5 Considerações finais

Ao longo deste bimestre, consolidei o entendimento prático sobre como projetar e estruturar sistemas distribuídos e resilientes em nuvem. O aprendizado foi além da containerização básica com Docker, abrangendo a separação real de domínios em microsserviços, o isolamento estrito de redes internas, o modelo de autorização RBAC com validação no servidor, a utilização de Redis Streams para trilhas de auditoria imutáveis de alta performance e a segregação de persistência entre bancos relacionais e Object Storage (MinIO). Compreender na prática os trade-offs entre diferentes decisões arquiteturais como centralização de autorização versus claims em tokens, ou bancos relacionais versus armazenamento de objetos foi o ponto alto do aprendizado.

O maior desafio técnico esteve na orquestração e sincronização da comunicação inter-serviços na malha interna do Docker, exigindo atenção constante no roteamento entre portas públicas e URLs privadas, além do tratamento de concorrência e inicialização assíncrona de dependências (como MariaDB, Redis e MinIO). Também demandou bastante esforço a implementação de camadas defensivas de segurança, como a inspeção de assinaturas binárias (Magic Bytes) para impedir MIME spoofing e a proteção rigorosa contra falhas de IDOR na edição de perfis.

Olhando para o trabalho realizado, o que eu faria diferente seria estabelecer contratos formais de API (como especificações OpenAPI/Swagger) e suítes de testes de integração automatizados desde a primeira atividade, o que teria agilizado a validação das rotas entre microsserviços durante as refatorações. Para o próximo bimestre, levo como base sólida a disciplina no isolamento de responsabilidades, o uso de variá-

veis de ambiente para portabilidade entre ambientes e a mentalidade de segurança por padrão (security by design) no desenvolvimento de aplicações escaláveis em nuvem.

6 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Felipe Yuzo Tanaka Shida

Pompeia, 07/10/2026